

PROGRAM 8

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~/oss
mkdir -p program8
cd program8
vedsk0831@DESKTOP-83FU7HB:~/oss/program8$ nano program8.c
vedsk0831@DESKTOP-83FU7HB:~/oss/program8$ gcc -Wall -Wextra -g program8.c -o program8
vedsk0831@DESKTOP-83FU7HB:~/oss/program8$ ./program8
```

```
=====
                OSSP PROGRAM 8
            Variable Expansion + Built-ins
=====
```

Type 'help' for commands.

```
shell8> shell8> echo $USER
```

Expanded tokens:

```
argv[0] = [shell8>]
argv[1] = [echo]
argv[2] = [vedsk0831]
```

```
execvp: No such file or directory
Child exited with status 1
```

```
shell8> shell8> echo $HOME
```

Expanded tokens:

```
argv[0] = [shell8>]
argv[1] = [echo]
argv[2] = [/home/vedsk0831]
```

```
execvp: No such file or directory
Child exited with status 1
```

```
shell8> shell8> echo $ABCXYZ
```

Expanded tokens:

```
argv[0] = [shell8>]
argv[1] = [echo]
argv[2] = []
```

```
execvp: No such file or directory
Child exited with status 1
```

```
shell8> pwd
```

Expanded tokens:

```
argv[0] = [pwd]
/home/vedsk0831/oss/program8
```

```
shell8> cd /tmp
```

Expanded tokens:

```
argv[0] = [cd]
argv[1] = [/tmp]
```

```
shell8> pwd
```

Expanded tokens:

```
argv[0] = [pwd]
/tmp
```

```
shell8> /tmp
```

Expanded tokens:

```
argv[0] = [/tmp]
```

```
execvp: Permission denied
Child exited with status 1
```

```
shell8> set NAME Vedika
```

```
Expanded tokens:
  argv[0] = [set]
  argv[1] = [NAME]
  argv[2] = [Vedika]
Variable NAME set to Vedika
```

```
shell8> echo $NAME
```

```
Expanded tokens:
  argv[0] = [echo]
  argv[1] = [Vedika]
Vedika
Child exited with status 0
```

```
shell8> getenv NAME
```

```
Expanded tokens:
  argv[0] = [getenv]
  argv[1] = [NAME]
NAME = Vedika
```

```
shell8> help
```

```
Expanded tokens:
  argv[0] = [help]

Built-in commands:
  cd <directory> - change directory
  pwd             - show current directory
  set NAME VALUE  - set environment variable
  getenv NAME     - display variable
  help           - show help
  exit           - exit program
```

```
shell8> ls
```

```
Expanded tokens:
  argv[0] = [ls]
snap-private-tmp
systemd-private-2ec48471c89144eeb97620895398644c-chrony.service-3dLDnY
systemd-private-2ec48471c89144eeb97620895398644c-systemd-logind.service-LvQJqA
Child exited with status 0
```

```
shell8> xyz123
```

```
Expanded tokens:
  argv[0] = [xyz123]
execvp: No such file or directory
Child exited with status 1
```

C PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#define MAX_INPUT 1024
```

```
#define MAX_ARGS 100
```

```
/* ----- Variable Expansion ----- */
```

```
char *expand_variable(const char *name)
```

```
{
```

```
    char *value = getenv(name);
```

```
    if (value == NULL)
```

```
        return strdup("");
```

```
    return strdup(value);
```

```
}
```

```
/* Expand $VARIABLE inside a token */
```

```
char *expand_token(const char *token)
```

```
{
```

```
    char result[MAX_INPUT];
```

```
    int i = 0;
```

```
    int j = 0;
```

```

while (token[i] != '\0' && j < MAX_INPUT - 1)
{
    if (token[i] == '$')
    {
        i++;

        char variable[256];

        int v = 0;

        while (token[i] != '\0' &&
            (
                (token[i] >= 'A' && token[i] <= 'Z') ||
                (token[i] >= 'a' && token[i] <= 'z') ||
                (token[i] >= '0' && token[i] <= '9') ||
                token[i] == '_'
            ))
        {
            variable[v++] = token[i++];
        }

        variable[v] = '\0';

        char *value = expand_variable(variable);

        for (int k = 0; value[k] != '\0' &&
            j < MAX_INPUT - 1; k++)
        {
            result[j++] = value[k];

```

```

    }

    free(value);
}
else
{
    result[j++] = token[i++];
}
}

result[j] = '\0';

return strdup(result);
}

/* ----- Parsing ----- */

int parse_command(char *input, char *args[])
{
    int argc = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && argc < MAX_ARGS - 1)
    {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }
}

```

```

    }

    args[argc] = NULL;

    return argc;
}

/* ----- Built-in Commands ----- */

void builtin_help()
{
    printf("\nBuilt-in commands:\n");
    printf(" cd <directory> - change directory\n");
    printf(" pwd          - show current directory\n");
    printf(" set NAME VALUE - set environment variable\n");
    printf(" getenv NAME    - display variable\n");
    printf(" help          - show help\n");
    printf(" exit          - exit program\n\n");
}

int builtin_pwd()
{
    char directory[MAX_INPUT];

    if (getcwd(directory, sizeof(directory)) == NULL)
    {

```

```
        perror("getcwd");
        return 1;
    }

    printf("%s\n", directory);

    return 0;
}

int builtin_cd(char *args[], int argc)
{
    if (argc < 2)
    {
        printf("Usage: cd <directory>\n");
        return 1;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
        return 1;
    }

    return 0;
}
```

```
int builtin_set(char *args[], int argc)
{
    if (argc < 3)
    {
        printf("Usage: set NAME VALUE\n");
        return 1;
    }

    if (setenv(args[1], args[2], 1) != 0)
    {
        perror("setenv");
        return 1;
    }

    printf("Variable %s set to %s\n",
        args[1], args[2]);

    return 0;
}
```

```
int builtin_getenv(char *args[], int argc)
{
    if (argc < 2)
    {
        printf("Usage: getenv NAME\n");
        return 1;
    }
}
```

```
char *value = getenv(args[1]);

if (value == NULL)
{
    printf("%s is undefined.\n", args[1]);
}
else
{
    printf("%s = %s\n",
        args[1], value);
}

return 0;
}
```

```
/* ----- Built-in Dispatcher ----- */
```

```
int handle_builtin(char *args[], int argc)
{
    if (argc == 0)
        return 1;

    if (strcmp(args[0], "help") == 0)
    {
        builtin_help();
        return 1;
    }
}
```

```
}

if (strcmp(args[0], "pwd") == 0)
{
    builtin_pwd();
    return 1;
}

if (strcmp(args[0], "cd") == 0)
{
    builtin_cd(args, argc);
    return 1;
}

if (strcmp(args[0], "set") == 0)
{
    builtin_set(args, argc);
    return 1;
}

if (strcmp(args[0], "getenv") == 0)
{
    builtin_getenv(args, argc);
    return 1;
}

return 0;
}
```

```

/* ----- Main ----- */

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    char *expanded[MAX_ARGS];

    printf("=====\n");
    printf("      OSSP PROGRAM 8\n");
    printf("  Variable Expansion + Built-ins\n");
    printf("=====\n");

    printf("\nType 'help' for commands.\n\n");

    while (1)
    {
        printf("shell8> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)

```

```

        continue;

    if (strcmp(input, "exit") == 0)
        break;

    int argc = parse_command(input, args);

    if (argc == 0)
        continue;

    /*
     * Expand variables in every token.
     */
    for (int i = 0; i < argc; i++)
    {
        expanded[i] = expand_token(args[i]);
    }

    expanded[argc] = NULL;

    /*
     * Display expanded tokens.
     */
    printf("\nExpanded tokens:\n");

    for (int i = 0; i < argc; i++)
    {
        printf("  argv[%d] = [%s]\n",

```

```

        i, expanded[i]);
    }

/*
 * Built-in command dispatch.
 */
if (handle_builtin(expanded, argc))
{
    for (int i = 0; i < argc; i++)
        free(expanded[i]);

    printf("\n");
    continue;
}

/*
 * External command:
 * Create child process.
 */
pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
}
else if (pid == 0)
{
    execvp(expanded[0], expanded);
}

```

```

        perror("execvp");
        exit(EXIT_FAILURE);
    }
    else
    {
        int status;

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status %d\n",
                WEXITSTATUS(status));
        }
    }

    /*
    * Free dynamically allocated tokens.
    */
    for (int i = 0; i < argc; i++)
        free(expanded[i]);

    printf("\n");
}

printf("Program 8 terminated.\n");

```

```
return 0;
```

```
}
```